DATABASE MANAGEMENT SYSTEM LAB

NAME: MONISHKA BAJAJ

ROLL No: 2401010161

LAB ASSIGNMENT-1

Q- Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COLLEGE DATABASE:

STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

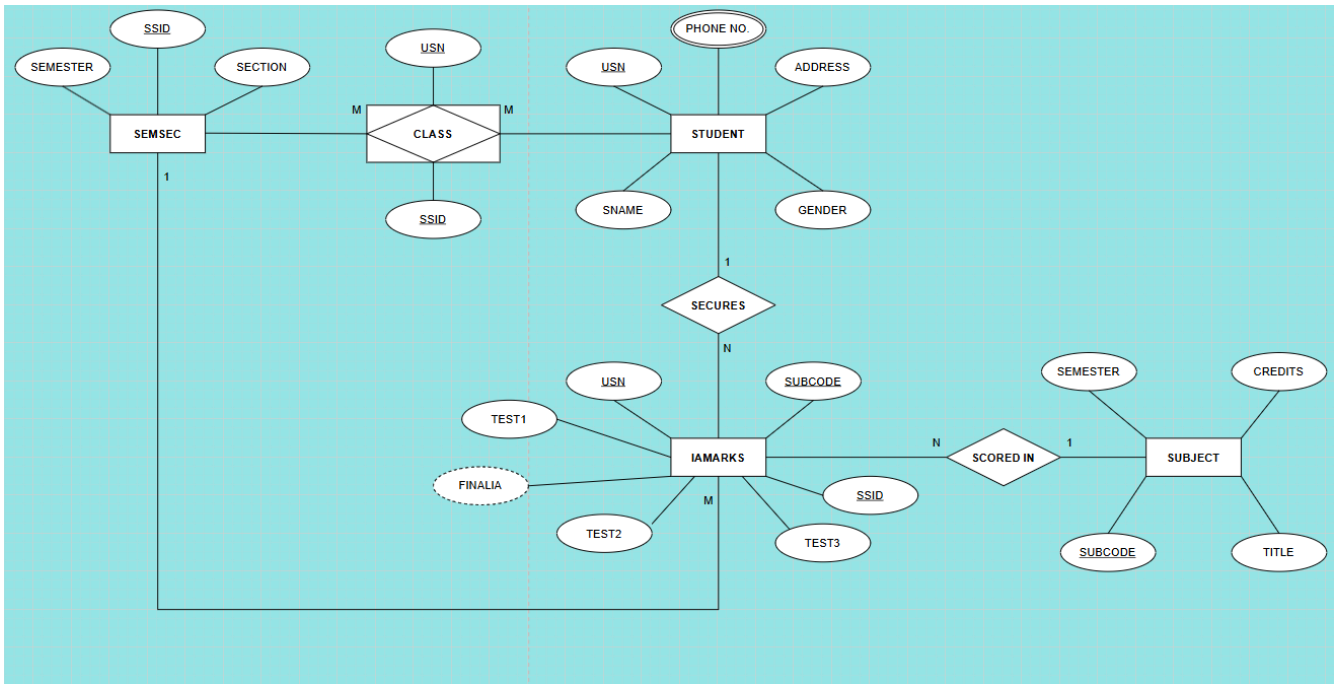
SUBJECT (Subcode, Title, Sem, Credits)

IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

ER DIAGRAM – COLLEGE DATABASE

DESCRIPTION

The ER diagram represents the **College Database System**. It is designed to store information about students, their classes, subjects, and internal assessment marks.



ENTITIES AND ATTRIBUTES

1. STUDENT Entity

The STUDENT entity stores details of students studying in the college.
It has the following attributes:

- **USN** (Primary Key)
- SName
- Address
- Phone No
- Gender

Each student is uniquely identified by the USN.

2. SEMSEC Entity

The SEMSEC entity represents the semester and section details.
It contains the following attributes:

- **SSID** (Primary Key)
- Semester
- Section

SSID uniquely identifies each semester–section combination.

3. CLASS Relationship

The CLASS relationship represents the association between STUDENT and SEMSEC.
It shows that a student belongs to a particular semester and section.

The attributes of CLASS are:

- USN
- SSID

This relationship helps in mapping students to their respective classes.

4. SUBJECT Entity

The SUBJECT entity stores information about subjects offered by the college.
The attributes are:

- **Subcode** (Primary Key)
- Title
- Semester

- Credits

Each subject is uniquely identified by its subject code.

5. IAMARKS Entity

The IAMARKS entity stores the internal assessment marks of students.
It includes the following attributes:

- USN
- Subcode
- SSID
- Test1
- Test2
- Test3
- FinalIA

IAMARKS records the marks obtained by a student in a particular subject.

RELATIONSHIP

1. SECURES Relationship

The SECURES relationship exists between STUDENT and IAMARKS.
It indicates that a student secures internal assessment marks for different subjects.

2. SCORED IN Relationship

The SCORED IN relationship connects IAMARKS and SUBJECT.
It shows that the internal assessment marks are scored for a specific subject.

CONCLUSION

Thus, the ER diagram clearly represents the entities, attributes, and relationships involved in the college database system and explains how students, classes, subjects, and internal assessment marks are related to each other.

LAB ASSIGNMENT-2

Q- Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COMPANY DATABASE:

EMPLOYEE (SSN, Name, Address, Sex, Salary, SuperSSN, DNo)

DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)

DLOCATION (DNo,DLoc)

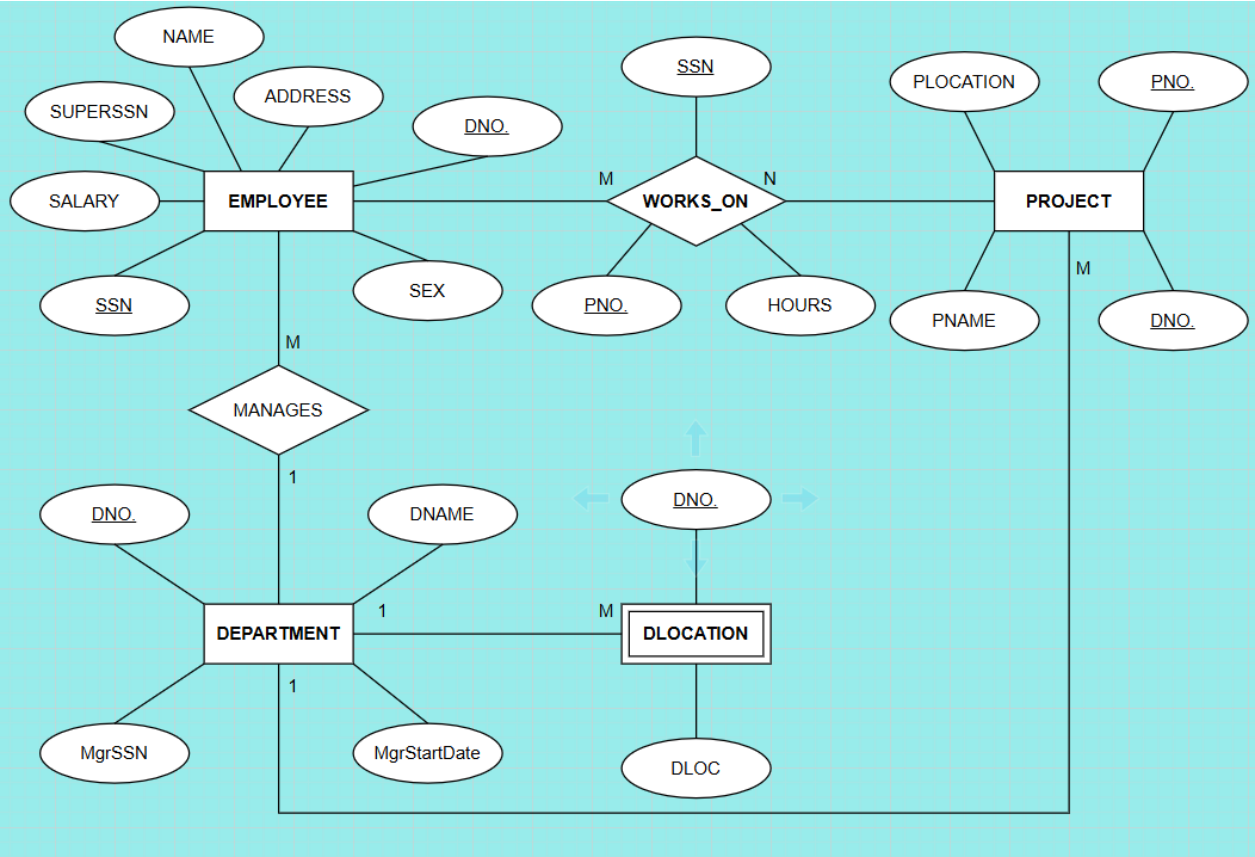
PROJECT (PNo, PName, PLocation, DNo)

WORKS_ON (SSN, PNo, Hours)

ER DIAGRAM – COMPANY DATABASE

DESCRIPTION

The given ER diagram represents a **Company Database System**. It describes the structure of the organization by showing entities, their attributes, and the relationships among them.



ENTITIES AND ATTRIBUTES

1. EMPLOYEE

The EMPLOYEE entity stores information about employees working in the company.

Attributes:

- **SSN** – Unique identifier for each employee (Primary Key)
- **Name** – Name of the employee
- **Address** – Residential address
- **Sex** – Gender of the employee
- **Salary** – Salary of the employee
- **SuperSSN** – SSN of the supervisor
- **DNO** – Department number in which the employee works

2. DEPARTMENT

The DEPARTMENT entity stores details about different departments in the company.

Attributes:

- **DNO** – Department number (Primary Key)
- **DNAME** – Name of the department
- **MgrSSN** – SSN of the department manager
- **MgrStartDate** – Date when the manager started managing the department

3. PROJECT

The PROJECT entity stores information about projects handled by the company.

Attributes:

- **PNO** – Project number (Primary Key)
- **PNAME** – Project name
- **PLOCATION** – Location of the project
- **DNO** – Department number that controls the project

4. DLOCATION

The DLOCATION entity represents the locations of departments.

Attributes:

- **DLOC** – Department location

- **DNO** – Department number

RELATIONSHIP

1. WORKS_ON

This relationship connects **EMPLOYEE** and **PROJECT** entities.
It shows which employee works on which project.

Attributes:

- **SSN** – Employee SSN
- **PNO** – Project number
- **HOURS** – Number of hours an employee works on a project

2. MANAGES

This relationship connects **EMPLOYEE** and **DEPARTMENT** entities.
It indicates that an employee manages a department.

CONCLUSION

This ER diagram provides a clear representation of the company's organizational structure, including employees, departments, projects, and their relationships. It helps in designing an efficient database for managing company operations.

LAB ASSIGNMENT-3

Q- Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

```sql
CREATE DATABASE customer_sale;
USE customer_sale;
```

1. Create the tables with the appropriate integrity constraints.

```sql
CREATE TABLE Customer (
    cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50) NOT NULL
);

CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50) NOT NULL,
    price INT NOT NULL CHECK (price > 0)
);

CREATE TABLE Sale (
    bill_no INT PRIMARY KEY,
    bill_date DATE NOT NULL,
    cust_id INT,
    item_id INT,
    qty_sold INT NOT NULL CHECK (qty_sold > 0),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

2. Insert around 10 records in each of the tables.

```sql
INSERT INTO Customer VALUES
(1,'Aman'),
(2,'Riya'),
(3,'Karan'),
(4,'Sneha'),
(5,'Rahul'),
(6,'Priya'),
(7,'Vikas'),
(8,'Neha'),
(9,'Arjun'),
(10,'Pooja');
INSERT INTO Item VALUES
(101,'Pen',20),
(102,'Notebook',50),
(103,'Bag',800),
(104,'Shoes',1200),
(105,'Watch',1500),
(106,'Bottle',150),
(107,'Headphones',700),
(108,'Keyboard',900),
(109,'Mouse',400),
(110,'Charger',300);
```

```sql
INSERT INTO Sale VALUES
(1001,CURDATE(),1,101,5),
(1002,CURDATE(),2,103,1),
(1003,CURDATE(),3,104,2),
(1004,CURDATE(),4,102,3),
(1005,CURDATE(),5,105,1),
(1006,CURDATE()-1,6,109,2),
(1007,CURDATE()-2,7,108,1),
(1008,CURDATE()-3,8,110,4),
(1009,CURDATE(),9,107,2),
(1010,CURDATE(),10,106,6);
```

3. List all the bills for the current date with the customer names and item numbers.

```sql
SELECT s.bill_no,
       s.bill_date,
       c.cust_name,
       s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows:

bill_no	bill_date	cust_name	item_id
1001	2026-02-22	Aman	101
1002	2026-02-22	Riya	103
1003	2026-02-22	Karan	104
1004	2026-02-22	Sneha	102
1005	2026-02-22	Rahul	105
1009	2026-02-22	Arjun	107
1010	2026-02-22	Pooja	106

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT s.bill_no,
       c.cust_name,
       i.item_name,
       s.qty_sold,
       i.price,
       (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

bill_no	cust_name	item_name	qty_sold	price	final_amount
1001	Aman	Pen	5	20	100
1002	Riya	Bag	1	800	800
1003	Karan	Shoes	2	1200	2400
1004	Sneha	Notebook	3	50	150
1005	Rahul	Watch	1	1500	1500
1006	Priya	Mouse	2	400	800
1007	Vikas	Keyboard	1	900	900
1008	Neha	Charger	4	300	1200
1009	Arjun	Headphones	2	700	1400
1010	Pooja	Bottle	6	150	900

5.List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT DISTINCT c.cust_id,
       c.cust_name
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

Result Grid | Filter Rows:

cust_id	cust_name
2	Riya
3	Karan
5	Rahul
9	Arjun
7	Vikas
6	Priya
8	Neha

6. Give a count of how many products have been bought by each customer

```
SELECT c.cust_id,
       c.cust_name,
       SUM(s.qty_sold) AS total_products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

Result Grid | Filter Rows:

cust_id	cust_name	total_products
1	Aman	5
2	Riya	1
3	Karan	2
4	Sneha	3
5	Rahul	1
6	Priya	2
7	Vikas	1
8	Neha	4
9	Arjun	2
10	Pooja	6

7. Give a count of how many products have been bought by each customer

```
SELECT i.item_name,
       s.qty_sold
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

Result Grid | Filter Rows:

item_name	qty_sold
Watch	1

8. List the item details which are sold as of today.

```sql
SELECT DISTINCT i.item_id,
        i.item_name,
        i.price
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows:

item_id	item_name	price
101	Pen	20
103	Bag	800
104	Shoes	1200
102	Notebook	50
105	Watch	1500
107	Headphones	700
106	Bottle	150

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```sql
CREATE VIEW Bill_Details AS
SELECT s.bill_no,
        s.bill_date,
        s.cust_id,
        s.item_id,
        i.price,
        s.qty_sold,
        (i.price * s.qty_sold) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
SELECT * FROM Bill_Details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1001	2026-02-22	1	101	20	5	100
1002	2026-02-22	2	103	800	1	800
1003	2026-02-22	3	104	1200	2	2400
1004	2026-02-22	4	102	50	3	150
1005	2026-02-22	5	105	1500	1	1500
1006	2026-02-21	6	109	400	2	800
1007	2026-02-20	7	108	900	1	900
1008	2026-02-19	8	110	300	4	1200
1009	2026-02-22	9	107	700	2	1400
1010	2026-02-22	10	106	150	6	900

10. Create a view which lists the daily sales date wise for the last one week

```
CREATE VIEW Weekly_Sales AS
SELECT s.bill_date,
       SUM(i.price * s.qty_sold) AS total_daily_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
SELECT * FROM Weekly_Sales;
```

Result Grid | Filter Rows:

bill_date	total_daily_sales
2026-02-22	7250
2026-02-21	800
2026-02-20	900
2026-02-19	1200